

LCD-bitmap IDE

LCD-bitmap IDE Operation User Manual

Beginner-friendly workflow for designing, checking and exporting embedded LCD measurement interfaces.

1. What This Application Does

2. Install And Start

3. Open The Demo Project

4. Understand The Workspace

5. Build A Measurement Workflow

6. Design LCD Screens

7. Bind Buttons, Tags And Procedures

8. Preview The Workflow

9. Export For Firmware

10. Save, Version And Share

11. Troubleshooting

1. What This Application Does

LCD-bitmap IDE is an offline desktop workbench for monochrome LCD screens, finite-state-machine navigation, control-panel bindings and firmware-ready exports.

Use it to design the operator interface around a measurement workflow: setup, start, progress, result review, warnings and service states.

The application does not perform physical measurements by itself. Real measurement acquisition requires your instrument firmware, backend service or hardware connector. In LCD-bitmap IDE you design and validate the user interface and its runtime behavior before integrating it with target firmware.

- ☐ Create or open a project.
- ☐ Design screens and FSM transitions.
- ☐ Preview the workflow.
- ☐ Export firmware assets.

2. Install And Start

1. Install Node.js 22 or newer and Git.
2. Clone the repository and run `npm ci`.
3. Start the browser build with `npm run dev`, or start the desktop build with `npm run electron:dev`.
4. Use the desktop build when you need REST API, MCP automation, PDF export or local app packaging.

```
git clone https://github.com/myahlovvlad/LCD-bitmap-IDE.git
cd LCD-bitmap-IDE
npm ci
npm run electron:dev
```

3. Open The Demo Project

On the start screen, choose the bundled demo or open examples/universal-lcd-demo.lcdproj.

The demo is intentionally generic. It shows how a measurement-oriented interface can be represented without vendor implementation files, private device identifiers or proprietary screen catalogs.

1. Open the demo.
2. Visit FSM, LCD, Tags and Runtime workspaces.
3. Do not edit everything at once. Change one screen or transition, then preview it.

4. Understand The Workspace

Workspace	Purpose	Beginner result
FSM	States and transitions	A clear operator flow
LCD	Pixel-accurate 128x64 screens	Readable screen layouts
Control panel	Button and event bindings	Predictable operator input
Tags	Named values used by screens and procedures	Traceable measurement values
Runtime	Interactive flow preview	A checked workflow before firmware export

5. Build A Measurement Workflow

Start with the states an operator expects, not with pixels. A simple workflow usually has Idle, Setup, Ready, Running, Result, Error and Service states.

Each state should answer one user question: what is happening, what value matters now, and what action is available.

1. Add the workflow states in FSM.
2. Name each transition with an event such as START, NEXT, CANCEL or RETRY.
3. Keep error transitions explicit.
4. Run the workflow in Runtime Preview before editing detailed graphics.

6. Design LCD Screens

Design for quick reading. Put the current state at the top, the key value in the center and the available action near the bottom.

Use text, lines, rectangles, bitmap layers and special elements such as progress bars or indicators. Avoid filling the screen with decorative content.

- Text is readable on 128x64.
- Important values have units.
- The active action is visible.
- Warning screens are visually distinct.

7. Bind Buttons, Tags And Procedures

Buttons should fire events that match FSM transitions. Tags should describe measurement values, configuration values or status flags. Procedures describe backend actions such as starting a reading, clearing a result or opening a service routine.

Use stable names. A clear tag such as `sample_absorbance` is easier to maintain than `value1`.

8. Preview The Workflow

1. Open Runtime.
2. Start from the initial state.
3. Press each expected button in the same order as a real operator.
4. Check that every measurement status screen appears in the right order.
5. Test error and cancel paths, not only the successful path.

9. Export For Firmware

After the workflow is stable, export screens as C headers, binary frame buffers, XBM, Arduino PROGMEM, Rust embedded-graphics or ESP-IDF assets.

Confirm that your target display uses the same resolution, page order, bit order and monochrome format before flashing hardware.

10. Save, Version And Share

- ❑ Save the .lcdproj file after meaningful changes.
- ❑ Commit project files and exported assets separately.
- ❑ Include screenshots in reviews.
- ❑ Keep generated firmware artifacts traceable to a project version.

11. Troubleshooting

Symptom	Likely cause	Action
Runtime button does nothing	No transition for that event	Check FSM transition event names
Export looks inverted	Target display bit order differs	Verify driver byte order
Text is clipped	Text object or font is too large	Use shorter copy or reposition the text
API connection refused	Desktop app is not running	Start npm run electron:dev or packaged app